Hands-on 4: Difference between JPA, Hibernate and Spring Data JPA

-Hemavarshini S

Introduction

JPA, Hibernate, and Spring Data JPA are related technologies used in Java applications for database operations. They help in storing, retrieving, updating, and deleting data from a database using Java objects. Although they are connected, each one has a different role.

1) JPA (Java Persistence API)

JPA stands for **Java Persistence API**. It is a **specification** provided by Java for object-relational mapping (ORM). JPA defines a standard way to map Java classes to database tables and perform database operations.

Features of JPA:

- It is only a **specification**, not an implementation.
- It uses annotations such as:
    - @Entity
    - @Table
    - @Id
    - @Column
- It provides a standard way to work with persistence in Java applications.
- It needs an implementation such as Hibernate to actually perform database operations.

Example from my project:

In the orm-learn project, the Country class uses JPA annotations to map the Java object to the country table in MySQL.

Example:

```
@Entity
@Table(name = "country")
public class Country {

  @Id
  @Column(name = "co_code")
  private String code;
```

```
    @Column(name = "co_name")
    private String name;
}
```

Here, JPA annotations are used to define how the Java class is connected to the database table.

2) Hibernate

Hibernate is an **ORM framework** and one of the most popular **implementations of JPA**. It actually performs the database operations based on the JPA configuration and annotations.

Features of Hibernate:

- It is a **framework/tool**.
- It implements JPA.
- It converts Java objects into database records and database records back into Java objects.
- It generates SQL queries automatically.
- It manages database interaction internally.

Example from my project:

In the orm-learn project, when the application ran and fetched the list of countries, Hibernate generated and executed the SQL query automatically in the background.

Example output:

select c1_0.co_code, c1_0.co_name from country c1_0

This shows that Hibernate handled the actual communication with the MySQL database.

3) Spring Data JPA

Spring Data JPA is a **higher-level abstraction** built on top of JPA. It reduces boilerplate code and makes database operations much easier by providing ready-made repository interfaces.

Features of Spring Data JPA:

- It does **not implement JPA** by itself.
- It works on top of JPA providers like Hibernate.
- It reduces the amount of code needed for database operations.
- It provides interfaces like JpaRepository.
- It automatically provides methods such as:
    - findAll()
    - findById()

- save()
- delete()

Example from my project:

In the orm-learn project, the CountryRepository interface extends JpaRepository.

Example:

```
public interface CountryRepository extends JpaRepository<Country, String> {

}
```

Because of Spring Data JPA, I did not need to write SQL queries or session handling code manually to fetch country data.

Difference between JPA, Hibernate and Spring Data JPA

Feature	**JPA**	**Hibernate**	**Spring Data JPA**
Type	Specification	Framework / ORM tool	Abstraction layer on top of JPA
Purpose	Defines standard rules for persistence	Implements JPA and performs ORM operations	Simplifies database access using repositories
Implementation	No implementation	Actual implementation	Uses JPA implementation internally
SQL Execution	No	Yes	Uses Hibernate/JPA provider for execution
Boilerplate Code	Moderate	More code if used directly	Very less code
Example in project	@Entity, @Table, @Id, @Column in Country.java	Generated SQL query during execution	CountryRepository extends JpaRepository

Comparison with coding style

Using Hibernate directly

If Hibernate is used directly, we need to write session and transaction code manually.

Example:

```
public Integer addEmployee(Employee employee){
   Session session = factory.openSession();
   Transaction tx = null;
```

```
    Integer employeeID = null;

    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    } catch (HibernateException e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeID;
}
```

This requires more code and manual transaction handling.

Using Spring Data JPA

With Spring Data JPA, the same task becomes much simpler.

Repository:

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

Service:

```
@Autowired
private EmployeeRepository employeeRepository;

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
```

This reduces code and improves readability.

How all three worked in my orm-learn project

In my orm-learn project:

- **JPA** was used in the Country entity class through annotations such as @Entity, @Table, @Id, and @Column.
- **Spring Data JPA** was used in CountryRepository by extending JpaRepository.
- **Hibernate** worked in the background to generate and execute the SQL query on the MySQL database.

Flow in my project:

1. Country.java maps Java object to database table using **JPA annotations**.
2. CountryRepository uses **Spring Data JPA** for repository methods.
3. **Hibernate** converts the method call into SQL and fetches data from MySQL.
4. The result is returned to the service and displayed in the console.

Conclusion

JPA, Hibernate, and Spring Data JPA are closely related but serve different purposes. JPA provides the standard specification, Hibernate implements that specification and performs the actual ORM operations, and Spring Data JPA makes database access easier by reducing boilerplate code. In the orm-learn project, all three were used together: JPA for entity mapping, Spring Data JPA for repository handling, and Hibernate for executing SQL queries in the background.